MODULE - 2

Searching & Sorting Algorithms

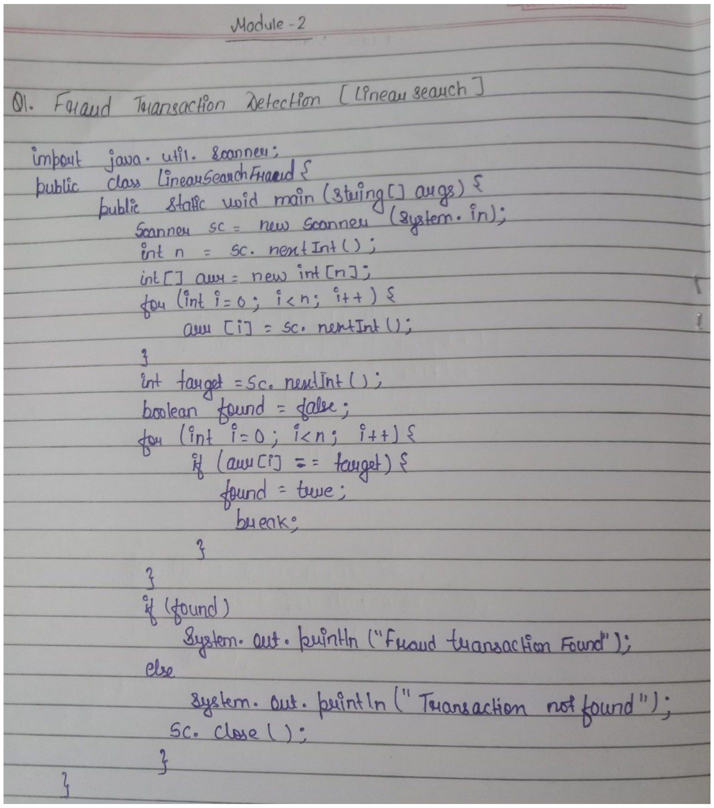
Name: Kumari Sapna

Course: B.Tech

Branch: AIML

Q1. Fraud Transaction Detection (Linear Search)

Problem: A bank stores transaction IDs in the order they occur. Since the transactions are not sorted, the security team needs to check if a suspicious transaction ID exists. Write a program using Linear Search.

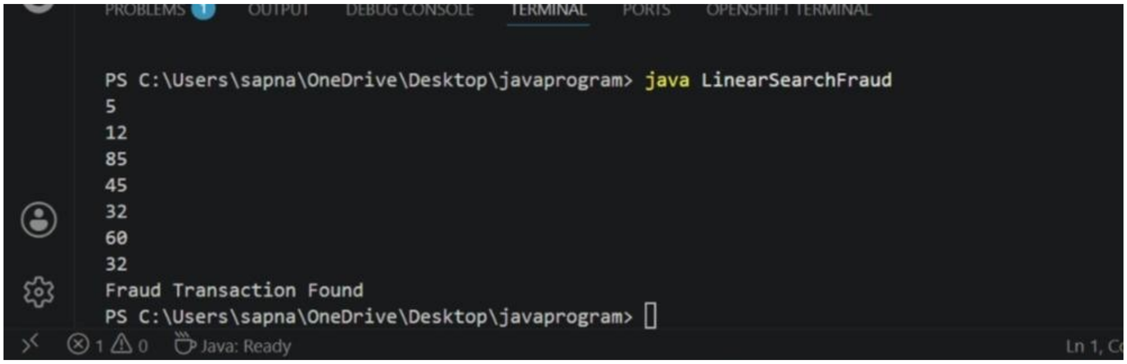


Module - 2

Q1. Fraud Transaction Detection [Linear search]

```
import java.util.Scanner;
public class LinearSearchFraud {
    public static void main (String[] args) {
        Scanner sc = new Scanner (System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i=0; i<n; i++) {
            arr[i] = sc.nextInt();
        }
        int target = sc.nextInt();
        boolean found = false;
        for (int i=0; i<n; i++) {
            if (arr[i] == target) {
                found = true;
                break;
            }
        }
        if (found)
            System.out.println("Fraud transaction Found");
        else
            System.out.println(" Transaction not found");
        sc.close();
    }
}
```

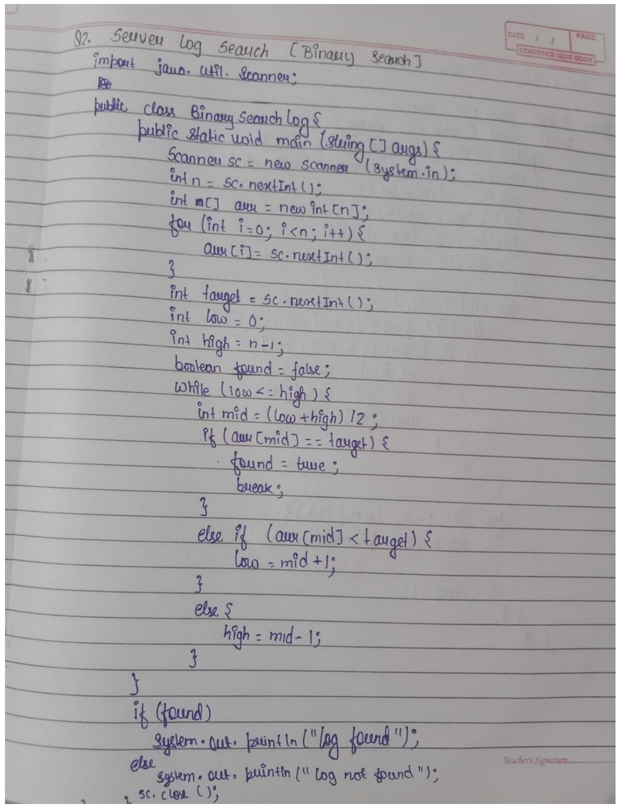
Output:



```
PS C:\Users\sapna\OneDrive\Desktop\javaprogram> java LinearSearchFraud
5
12
85
45
32
60
32
Fraud Transaction Found
PS C:\Users\sapna\OneDrive\Desktop\javaprogram>
```

Q2. Server Log Search (Binary Search)

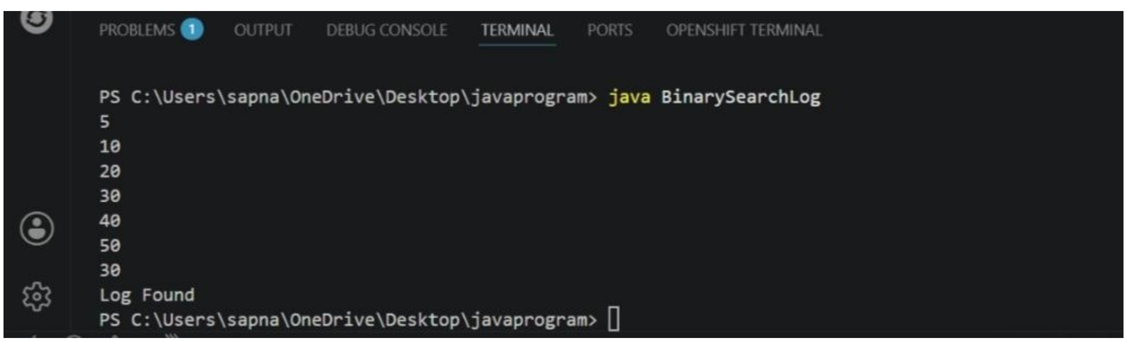
Problem: A cloud company stores server log IDs in sorted order. Engineers need to quickly verify if a log ID exists. Write a program using Binary Search.



```
Q2. Server Log Search [Binary Search]
import java.util.Scanner;

public class BinarySearchLog {
    public static void main (String[] args) {
        Scanner sc = new Scanner (System.in);
        int n = sc.nextInt();
        int [] arr = new int [n];
        for (int i=0; i<n; i++) {
            arr[i] = sc.nextInt();
        }
        int target = sc.nextInt();
        int low = 0;
        int high = n-1;
        boolean found = false;
        while (low <= high) {
            int mid = (low+high)/2;
            if (arr[mid] == target) {
                found = true;
                break;
            }
            else if (arr[mid] < target) {
                low = mid+1;
            }
            else {
                high = mid-1;
            }
        }
        if (found)
            System.out.println("Log found");
        else
            System.out.println("Log not found");
        sc.close();
```

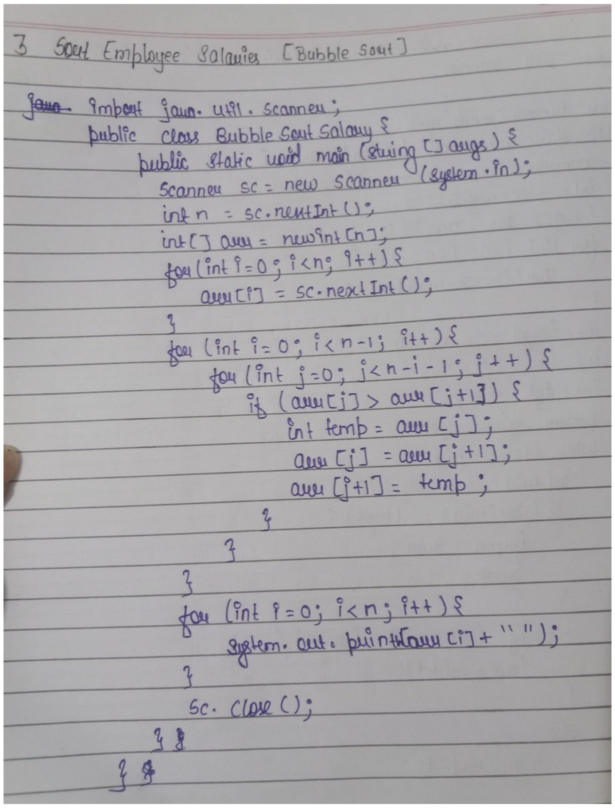
Output:




```
PS C:\Users\sapna\OneDrive\Desktop\javaprogram> java BinarySearchLog
5
10
20
30
40
50
30
Log Found
PS C:\Users\sapna\OneDrive\Desktop\javaprogram>
```

Q3. Sort Employee Salaries (Bubble Sort)

Problem Statement: A company wants to sort employee salaries in ascending order for payroll analysis. Write a program using Bubble Sort.



3 Sort Employee Salaries [Bubble Sort]

```
~~java~~ import java.util.Scanner;
public class BubbleSortSalary {
public static void main (String [] args) {
  Scanner sc = new Scanner (System.in);
  int n = sc.nextInt ();
  int[] arr = new int [n];
  for (int i=0; i<n; i++) {
      arr[i] = sc.nextInt ();
  }
  for (int i = 0; i<n-1; i++) {
    for (int j=0; j<n-i-1; j++) {
       if (arr[j] > arr[j+1]) {
          int temp = arr [j];
          arr [j] = arr [j+1];
          arr [j+1] = temp ;
       }
    }
  }
  for (int i=0; i<n; i++) {
     System.out.print(arr [i] + " ");
  }
  sc.close ();
}
}
```

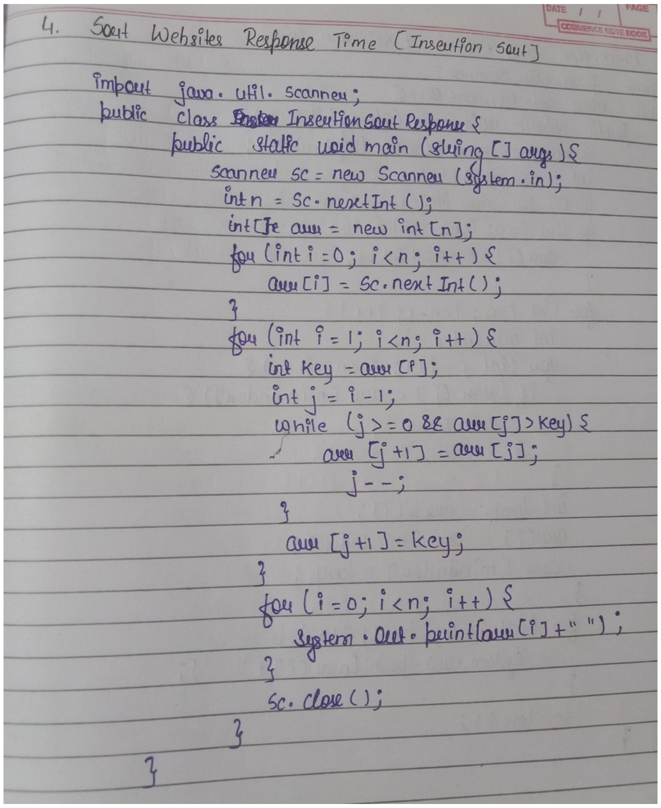
Output:

```
PS C:\Users\sapna\OneDrive\Desktop\javaprogram> Javac BubbleSortSalary.java
PS C:\Users\sapna\OneDrive\Desktop\javaprogram> java BubbleSortSalary

4
50000
30000
40000
70000
30000 40000 50000 70000
PS C:\Users\sapna\OneDrive\Desktop\javaprogram>
```

Q4. Sort Website Response Time (Insertion Sort)

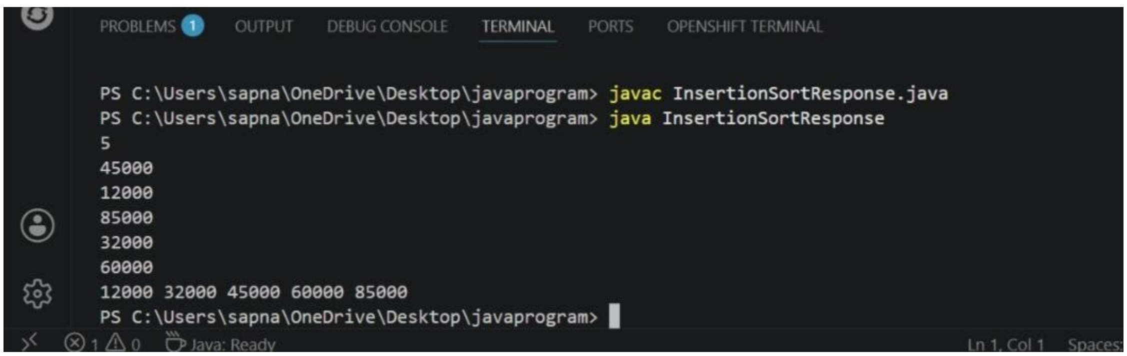
Problem Statement: A web monitoring system records website response times. New data comes continuously and must be inserted into the correct position. Write a program using Insertion Sort.



```
4. Sort Websites Response Time [Insertion Sort]

import java.util.Scanner;
public class InsertionSortResponse {
  public static void main (String [] args) {
    Scanner sc = new Scanner (System.in);
    int n = sc.nextInt ();
    int[] arr = new int [n];
    for (int i = 0; i<n; i++) {
      arr [i] = sc.nextInt();
    }
    for (int i = 1; i<n; i++) {
      int key = arr [i];
      int j = i - 1;
      while (j >= 0 && arr [j] > key) {
        arr [j+1] = arr [j];
        j--;
      }
      arr [j+1] = key;
    }
    for (i = 0; i<n; i++) {
      System.out.print(arr[i] + " ");
    }
    sc.close ();
  }
}
```

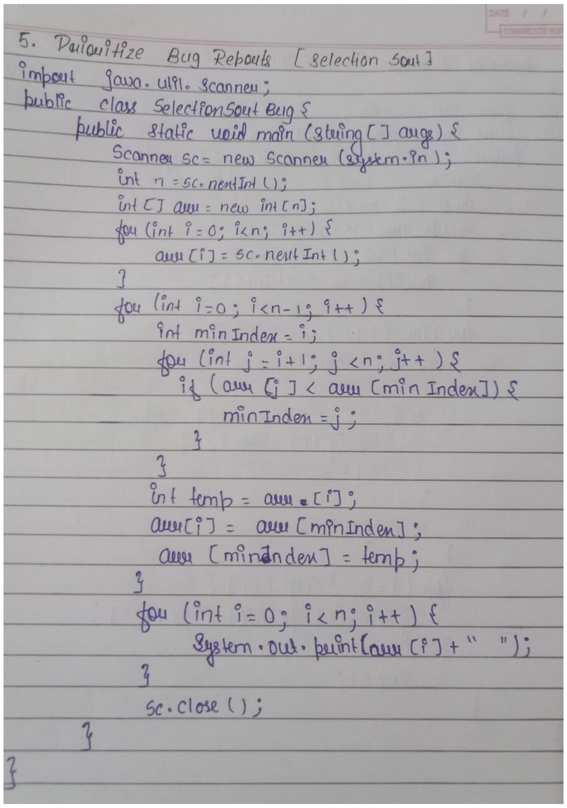
Output:



```
PS C:\Users\sapna\OneDrive\Desktop\javaprogram> javac InsertionSortResponse.java
PS C:\Users\sapna\OneDrive\Desktop\javaprogram> java InsertionSortResponse
5
45000
12000
85000
32000
60000
12000 32000 45000 60000 85000
PS C:\Users\sapna\OneDrive\Desktop\javaprogram>
```

Q5. Prioritize Bug Reports (Selection Sort)

Problem Statement: A software company assigns priority scores to bug reports. To resolve bugs efficiently, they want to sort priorities in ascending order. Write a program using Selection Sort.



```
5. Prioritize Bug Reports [Selection Sort]
import java.util.Scanner;
public class SelectionSortBug {
    public static void main (String[] args) {
        Scanner sc = new Scanner (System.in);
        int n = sc.nextInt ();
        int[] arr = new int[n];
        for (int i = 0; i<n; i++) {
            arr[i] = sc.nextInt ();
        }
        for (int i=0; i<n-1; i++) {
            int minIndex = i;
            for (int j = i+1; j<n; j++ ) {
                if (arr [j] < arr [minIndex]) {
                    minIndex = j;
                }
            }
            int temp = arr[i];
            arr[i] = arr [minIndex];
            arr [minIndex] = temp;
        }
        for (int i = 0; i<n; i++) {
            System.out.print(arr [i] + " ");
        }
        sc.close ();
    }
}
```

Output: